security, attendance management, or personal use, by leveraging state-of-the-art face recognition technologies.

Detailed Plan:

1. Face Detection:
    - Use OpenCV's `cv2.CascadeClassifier` to detect faces in real-time video frames.
    - Employ pre-trained Haar cascades or DNN-based detectors for enhanced accuracy and performance.
    - Ensure the system is optimized for different lighting conditions and camera angles.

2. Feature Extraction:
    - Integrate deep learning models like FaceNet or dlib's ResNet-based face recognition module for feature extraction.
    - Generate 128-dimensional face embeddings to uniquely identify individuals.
    - Store these embeddings in a database or file system for comparison.

3. Face Recognition:
    - Use a similarity metric (e.g., cosine similarity or Euclidean distance) to match real-time face embeddings with stored embeddings.
    - Implement multi-threading or asynchronous processing to enhance real-time performance.

4. Performance Optimization:
    - Leverage GPU acceleration using libraries like CUDA for faster frame processing.
    - Optimize frame rates to ensure smooth operation without sacrificing accuracy.